



# Bug Bounty Report

Missing Sequence Filter in AA Balance Initialization

Allows Final-Bad Outputs to Inflate Counterstake

Assistant Shares and Steal User Deposits

*alva-p.xyz*

May 2026

---

Prepared by: alva-p  
Lead Researcher: Pineda Alvaro

## Table of Contents

- Summary
- Root Cause
- Data Flow: From Final-Bad Output to Phantom Balance
- Attack Path
- Exploit Window and Timing
- Impact: In-Scope Code Patterns
- Full Attack Scenario
- Economic Model
- Proof of Concept
- Fix Direction
- Honest Report Boundary
- Timeline

---

## Summary

`storage.insertAADefinitions()` initializes a newly defined Autonomous Agent's (AA) `aa_balances` from historical outputs without filtering `units.sequence='good'`. A **final-bad** double-spend output sent to a predictable future Counterstake assistant address is therefore credited as real AA balance even though the AA composer cannot spend it. Counterstake assistants use `balance[$stake_asset]` to mint/redeem shares, so the first depositor can receive shares backed by phantom funds and later redeem those shares against honest users' real deposits.

**Severity:** Low

**Impact:** Direct theft of user funds deposited into a factory-created Counterstake assistant

**Status:** Confirmed and fixed

---

---

## Root Cause

File: `storage.js:943-948`

```
1 -- Vulnerable initialization query (storage.js:902 --
  insertAADefinitions())
2 INSERT INTO aa_balances (address, asset, balance)
3 SELECT address, IFNULL(asset, 'base'), SUM(amount) AS balance
4 FROM outputs CROSS JOIN units USING(unit)
5 WHERE address=? AND is_spent=0
6     AND (main_chain_index<? <or_sent_by_aa>)
7 GROUP BY address, asset
```

The query selects all unspent outputs sent to `address` with `main_chain_index < mci`. It joins `outputs` with `units` but **does not add** `AND units.sequence='good'` or `AND units.is_stable=1`.

In contrast, the AA payment composer's input selection query at `aa_composer.js:1071-1079` explicitly requires both:

```
1 -- Correct spend path (aa_composer.js:1071-1079 --
  readStableOutputs())
2 SELECT unit, message_index, output_index, amount, output_id
3 FROM outputs CROSS JOIN units USING(unit)
4 WHERE address=? AND asset ... AND is_spent=0
5     AND sequence='good' AND main_chain_index<=?
```

The divergence creates a gap:

---

| Path | Includes final-bad outputs? |
|------|-----------------------------|
|------|-----------------------------|

---

|                                    |           |
|------------------------------------|-----------|
| <code>insertAADefinitions()</code> | Yes (bug) |
|------------------------------------|-----------|

( ) – initial

`aa_balances`

|                          |              |
|--------------------------|--------------|
| <code>aa_composer</code> | No (correct) |
|--------------------------|--------------|

– inputs

for response

transaction

---

| Path | Includes final-bad outputs? |
|------|-----------------------------|
|------|-----------------------------|

---

|         |                        |
|---------|------------------------|
| Oscript | Yes (inherits the bug) |
|---------|------------------------|

`balance[]`  
getter –  
reads from  
`aa_balances`

---

So `balance[$asset]` inside an Oscript formula can return a value that includes phantom funds the AA can never actually spend.

## How `balance[]` Is Populated at Execution Time

At AA trigger time, `aa_composer.js:472-490` (`updateInitialAABalances`) reads the current `aa_balances` rows and places them into `objValidationState.assocBalances`. The Oscript evaluation engine reads these values when a formula references `balance[$asset]`. There is no re-check against actual spendable outputs – it trusts the `aa_balances` table entirely.

---

## Data Flow

```
1 Attacker signs unit U_bad
2   -> output of PHANTOM bytes -> FUTURE_AA_ADDRESS
3   -> another unit U_good spends the same UTXO (double-spend)
4
5 U_good becomes stable with sequence='good'
6 U_bad becomes stable with sequence='final-bad'
7   -> writer.saveJoint() persists U_bad and its outputs table row
8   -> archiving will eventually delete outputs of U_bad,
9       but this happens AFTER the AA definition is inserted
10
11 Factory defines the assistant;
12 ocore processes the definition through
13   insertAADefinitions(FUTURE_AA_ADDRESS, def, trigger_unit, mci)
14   -> initialization query runs:
15       SELECT SUM(amount) FROM outputs CROSS JOIN units
16       WHERE address=FUTURE_AA_ADDRESS
17       AND is_spent=0 AND main_chain_index < mci
18   -> picks up the U_bad output (is_spent=0, no sequence filter)
```

---

```
19  -> INSERT INTO aa_balances (FUTURE_AA_ADDRESS, 'base', PHANTOM)
20
21  AA executes:
22    Oscript balance['base'] = PHANTOM + real_trigger_amount
23    AA cannot spend PHANTOM (composer rejects final-bad outputs)
24    But share-mint formula uses balance['base'] to price new shares
25    -> overmint to attacker
```

---

## Attack Path

### Step 1 – Predict the Future AA Address

For factory-created AAs, the assistant address is deterministic:

```
1  var assistant_def = ['autonomous agent', {
2    base_aa: BASE_AA_ADDR,
3    params: { bridge_aa, manager, management_fee, success_fee }
4  }];
5  var future_address = chash160(assistant_def);
```

All inputs to `chash160` are public in the factory flow: the base AA address is fixed per bridge type, and the params are visible in the factory call transaction **before** the assistant definition becomes stable.

### Step 2 – Create the Poison Unit

Build and sign a payment unit that:

- Spends an existing UTXO
- Sends `PHANTOM` bytes to `future_address`
- Is broadcast concurrently with a conflicting spend of the same UTXO

### Step 3 – Let the Poison Branch Become Final-Bad

The winning conflicting unit stabilizes first. The losing branch's unit stabilizes as `sequence='final-bad'`. `writer.saveJoint()` persists it and its unspent output.

---

## Step 4 – Wait for the Factory to Define the AA

`insertAADefinitions()` runs without a `sequence='good'` guard and credits the phantom output.

## Step 5 – Deposit First

The attacker triggers the assistant with a small real deposit. The share-mint formula executes using `balance[$stake_asset]`, which now includes the phantom amount. The attacker receives shares backed by phantom + real deposit – a significant overmint.

## Step 6 – Wait for Honest Users to Deposit

Their real funds inflate the AA's actual (spendable) balance.

## Step 7 – Redeem

The attacker redeems over-allocated shares against the AA's real balance, extracting more than they deposited.

---

## Exploit Window and Timing

The critical ordering:

```
1 T0: U_bad becomes final-bad and stable -> outputs row written
2 T1: Factory fires -> insertAADefinitions() runs at T1 MCI
3     -> initialization query at T0 MCI < T1 MCI -> picks up phantom
    output
4     -> aa_balances poisoned [CONFIRMED]
5 T2: Network archives U_bad -> outputs row deleted
6     -> AFTER aa_balances already written; deletion has no effect
```

Because `insertAADefinitions()` runs **before** the archive pass for the bad unit, the window is sufficient. Deleting the source output does not trigger a compensating delete in `aa_balances`.

---

## Impact

The following AAs use `balance[]` in economically critical paths:

### Counterstake

`export-assistant.oscript` – computes gross balance for share minting/redemption:

```
1 $gross_balance = balance[$stake_asset] + $balance_in_work
2               - $received_stake_amount;
3 $balance = $gross_balance - $mf - $sf;
4 $coef = $shares_supply ? $shares_supply / $get_shares($balance) :
5       1;
6 $new_shares_supply = floor($coef
7                           * $get_shares($balance +
8                                       $received_stake_amount));
```

`import-assistant.oscript` – same pattern for both stake and image assets.

Factory scripts create assistants with **deterministic addresses**:

```
1 $assistant_aa = ['autonomous agent', {
2     base_aa: $assistant_base_aa, params: $params
3 }];
4 $assistant_aa_address = chash160($assistant_aa);
```

Strongest exposure in Counterstake because factory-created addresses are fully predictable before deployment and the share-mint formula directly uses `balance[$stake_asset]` for pricing.

---

## Full Attack Scenario

**Prerequisites:** attacker has a funded wallet; a new export-assistant is being deployed via the factory; attacker observes the factory call transaction before it stabilizes.

### Step 1 – Predict the assistant address

```
1 var FACTORY_PARAMS = {
2     bridge_aa:      '<observed bridge AA address>',
3     manager:        ATTACKER_ADDR,
4     management_fee: 0.01,
5     success_fee:     0.05
6 };
7 var FACTORY_AA_ADDR = objectHash.getChash160(
```

---

```

8      ['autonomous agent', {
9          base_aa: '<export-assistant base AA address>',
10         params:  FACTORY_PARAMS
11     }]
12 );

```

## Step 2 – Broadcast the poison unit

```

1 inputs:  [attacker_utxo]
2 outputs: [{ address: future_address, amount: 500,000 },
3           { address: attacker_address, amount: change }]

```

Simultaneously broadcast a conflicting unit spending the same UTXO.

**Step 3 – Losing branch finalizes as final-bad.** Both units persisted by `writer.saveJoint()`.

**Step 4 – Trigger the factory.** `insertAADefinitions()` runs and inserts:

```

1 INSERT INTO aa_balances (address, asset, balance)
2 VALUES ('<FACTORY_AA_ADDR>', 'base', 500000)

```

## Step 5 – Attacker deposits first (50,000 bytes)

```

1 gross_balance = 500,000 + 50,000 = 550,000
2 shares_supply = 0  ->  new_shares = 550,000
3 (correct would be 50,000 -- 11x overmint)

```

## Step 6 – Honest user deposits 800,000 bytes

```

1 new_shares = floor(550,000 * 1,350,000 / 550,000) = 1,350,000
2 Total shares = 1,900,000

```

## Step 7 – Attacker redeems 550,000 shares

Real spendable balance = 850,000 bytes:

```

1 attacker_payout = floor(850,000 * 550,000 / 1,900,000) = 246,052
  bytes

```

---

|                         |                                     |
|-------------------------|-------------------------------------|
| Attacker deposited      | 50,000 bytes                        |
| Attacker withdrew       | 246,052 bytes                       |
| <b>Net profit</b>       | <b>196,052 bytes (~392% return)</b> |
| Honest user deposited   | 800,000 bytes                       |
| Honest user can recover | ~603,948 bytes                      |

---

---

|                  |                                   |
|------------------|-----------------------------------|
| Honest user loss | 196,052 bytes (~24.5% of deposit) |
|------------------|-----------------------------------|

---

## Economic Model

| Phantom   | Attacker deposit | Honest deposit | Attacker profit | Honest loss |
|-----------|------------------|----------------|-----------------|-------------|
| 100,000   | 10,000           | 150,000        | 14,230          | 14,230      |
| 500,000   | 50,000           | 800,000        | 196,052         | 196,053     |
| 1,000,000 | 100,000          | 2,000,000      | 476,190         | 476,190     |

## Proof of Concept

**PoC repository:** <https://gist.github.com/alva-p/116a87a70a32f91f01a7b376eb0b52a7>

Run command:

```
1 npx ava \  
2   test/poc_aa_balance_final_bad.test.js \  
3   test/poc_aa_balance_final_bad_writer.test.js \  
4   test/poc_aa_balance_share_inflation.test.js \  
5   test/poc_counterstake_factory.test.js \  
6   --timeout=90s --concurrency=1 --fail-fast --verbose
```

Expected output:

```
1 [PASS] final-bad output is incorrectly credited to newly defined AA  
   balance  
2 [PASS] writer persists validator-produced final-bad output and  
3   insertAADefinitions imports it into aa_balances  
4 [PASS] phantom AA balance inflates first depositor shares and lets  
   them  
5   redeem later deposits  
6 [PASS] STAGE-1: attacker can precompute factory assistant address  
7   before factory fires  
8 [PASS] STAGE-2: validator marks poison unit final-bad; writer  
   persists
```

---

```
9           phantom output to factory address
10  [PASS] STAGE-3: factory insertAADefinitions credits phantom balance
11           ;
12           attacker inflates shares and profits
13  6 tests passed
```

## PoC 1 – Core DB-level Bug (poc\_aa\_balance\_final\_bad.test.js)

Minimal reproduction: inserts a stable final-bad unit directly into the test DB, calls `insertAADefinitions()`, checks `aa_balances`.

```
1  t.is(balances.base, phantom_amount);           // 100,000
2  t.true(arrResponses[0].bounced);
3  t.regex(arrResponses[0].response.error, /not enough funds/);
```

## PoC 2 – Full Validator/Writer Path (poc\_aa\_balance\_final\_bad\_writer.test.js)

Real secp256k1 key material and real signed unit:

```
1  t.is(res.result, 'ok');
2  t.is(res.state.sequence, 'final-bad');
3
4  var assocBalances = await storage.readAABalances(db, FUTURE_AA);
5  t.is(assocBalances.base, PHANTOM_AMOUNT);      // 100,000
```

## PoC 3 – Share Inflation Model (poc\_aa\_balance\_share\_inflation.test.js)

Full share-mint/redeem cycle through `aa_composer.estimatePrimaryAATrigger()`:

```
1  assocBalances[aa_address] = { base: 100000 }; // phantom pre-
    loaded
2
3  t.is(buy1[0].response.responseVars.shares, 110000); // 100,000 +
    10,000
4  t.true(withdrawn > 10000);                       // 114,230 >
    10,000
```

Console output:

```
1  attacker deposited 10000 and withdrew 114230
```

---

## PoC 4 – Counterstake Factory Path (poc\_counterstake\_factory.test.js)

Three stages – address prediction, writer-path poisoning, inflation.

```
1 // STAGE-1: addresses match
2 t.is(attacker_computed, FACTORY_AA_ADDR);
3
4 // STAGE-2: final-bad output persisted
5 t.is(res.state.sequence, 'final-bad');
6 t.is(out_row.amount, 500000);
7
8 // STAGE-3: phantom credited, shares inflated
9 t.is(assocBalances.base, 500000);
10 t.true(attackerPayout > 50000); // 246,052 > 50,000
```

Full output from STAGE-3:

```
1 Phantom balance at AA creation: 500000 bytes
2 Attacker deposits 50000 -> receives 550000 shares
3 (correct would be 50000 without phantom)
4 Honest user deposits 800000 -> receives 1350000 shares
5 Attacker redeems 550000/1900000 shares
6 Real balance (spendable): 850000 bytes
7 Attacker receives: 246052 bytes
8 Attacker profit: 196052 bytes
9 Honest user loss: 196053 bytes
```

---

## Fix Direction

### Primary Fix – storage.js:943-948

```
1 WHERE address=? AND is_spent=0
2 - AND (main_chain_index<? " + or_sent_by_aa + ")
3 + AND sequence='good' AND is_stable=1
4 + AND (main_chain_index<? " + or_sent_by_aa + ")
```

### Secondary – or\_sent\_by\_aa Branch

Apply the same `sequence='good'` filter to the alternative AA-authored branch for defense in depth.

---

## Consistency Note

The spend path in `aa_composer.js:1071-1079` already correctly requires `sequence='good'` AND `main_chain_index<=?`. The fix aligns initialization with spend.

---

## Honest Report Boundary

### What Is Directly Proven

- `storage.insertAADefinitions()` credits final-bad outputs (no `sequence='good'` filter)
- Real secp256k1-signed unit passes `validation.validate()` as `sequence='final-bad'`
- `writer.saveJoint()` persists the final-bad unit and its `outputs` row
- `insertAADefinitions()` imports that output into `aa_balances`
- Oscript `balance[]` reads the inflated value; `aa_composer` cannot spend it
- Factory-style AA address is deterministically computable before factory fires
- Share-mint formula overmints and enables profit extraction against the poisoned balance

### What Is Modeled but Not End-to-End

- **Double-spend race not modeled:** stable-good competitor inserted via DB fixture, not a second signed+validated unit
  - **Factory Oscript not executed:** `insertAADefinitions()` called directly from JS
  - **Economic model in JavaScript:** share formula ported from Oscript; math is identical
- 

## Timeline

---

| Date         | Event                                         |
|--------------|-----------------------------------------------|
| May 25, 2026 | Report submitted                              |
| May 25, 2026 | Escalated to project for technical assessment |
| May 26, 2026 | Bug confirmed by project team                 |

---

---

| Date         | Event                                     |
|--------------|-------------------------------------------|
| May 26, 2026 | Severity assessed as Low                  |
| May 26, 2026 | Bounty paid                               |
| May 26, 2026 | Permission granted to publish this report |